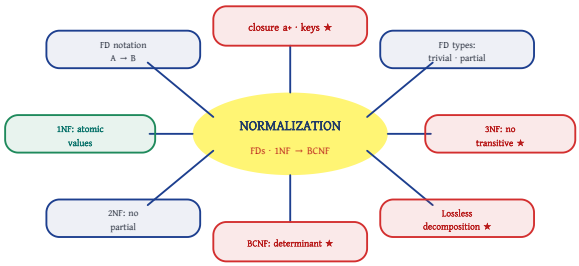
Functional Dependencies & Normalization

FD notation · attribute closure · 1NF → 2NF → 3NF → BCNF · lossless decomposition: exam-ready in 11 sheets



↑ the whole topic: red = high-yield, green = the first rung

Syllabus coverage

The Database Systems syllabus line in full: "**Normalization — Process of reducing redundancy and improving data consistency. Steps: 1NF → 2NF → 3NF → BCNF.**" **This lesson takes that clause in full.** The algebra and SQL that read these tables were covered in **db2**; the ER and relational models, file organisation, indexing and optimisation each have their own place in the chapter.

High-yield list

① `A → B`: same A ⇒ same B
② Trivial: Y ⊆ X · non-trivial: Y ⊄ X
③ Closure X^+ = everything from X
④ Superkey = any key; candidate = minimal
⑤ 1NF: atomic values, no repeating group
⑥ 2NF: no partial dependency
⑦ 3NF: no transitive dependency
⑧ **BCNF: every determinant is a key**
⑨ Lossless ⟺ **(R1∩R2)→R1** or **→R2**
⑩ Normalise for writes, not reads ★

Legend: **yellow** = key term · **red circle** = must-memorise · **purple** = memory trick · **red** = exam trap · **green** = worked example

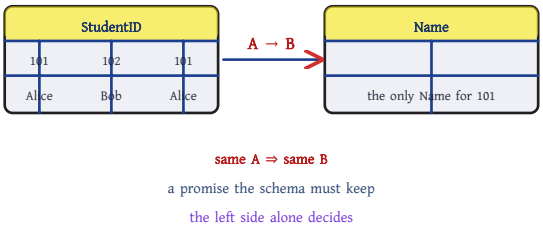
Contents

✓ FDs: notation and the five types … p. 2
✓ Attribute closure, keys … p. 3
✓ 1NF and 2NF … p. 4
✓ 3NF and BCNF … p. 5
✓ The ladder in one table … p. 6
✓ Lossless join, dependency preservation … p. 7
✓ Full normalization, step by step … p. 8
✓ Anomalies & denormalization … p. 9
✓ Night-before sheet … p. 10
✓ Practice MCQs + written … p. 11

1 Functional Dependencies: What A → B Really Means

Definition: R satisfies the functional dependency `X → Y` when **every two tuples that agree on X also agree on Y.** X is the **determinant**, Y the **dependent**; read `A → B` as "A determines B". Example: `StudentID → Name` — knowing the ID settles the name, so two rows sharing an ID can never carry different names. An FD is a statement about what a relation *means*, not about today's rows.

Type	Rule	Example	Note
Trivial	Y ⊆ X	`(StudentID, Name) → Name`	**always true**
Non-trivial	Y ⊄ X	`StudentID → Name`	the only ones that matter
Full	Y needs the whole of X	`(StudentID, CourseID) → Grade`	what 2NF insists on
Partial	Y needs only part of X	`StudentID → StudentName`, key (StudentID, CourseID)	**breaks 2NF**
Transitive	A→B and B→C give A→C	`StudentID → DeptID → DeptName`	**breaks 3NF**



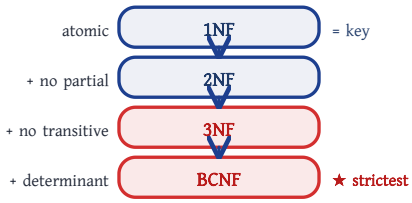
rows 1 and 3 share ID 101, so they must share the name

Memory trick: **"An arrow is a promise, not a coincidence."** If `A → B`, the database must *never* hold two rows with the same A and different B — that is why `StudentID` is a key candidate, and why "same student, two names" is a **design error**.

Exam trap ✍ two: `1. (A, B) → A` is **trivial** and true in every relation — no information at all. **2.** The arrow is **one-way**: `StudentID → Name` never licenses `Name → StudentID`, because two students may share a name.

Normalization: **Normalization** is the process of **reducing redundancy** and **improving data consistency** by decomposing a badly designed relation into smaller ones, each obeying a **normal form**. The ladder is cumulative: **1NF → 2NF → 3NF → BCNF**, each rung adding one condition to the one below.

Why bother?: A relation in BCNF stores each fact in **exactly one place**, which removes the three anomalies — update, insert and delete. The price is extra joins at read time.



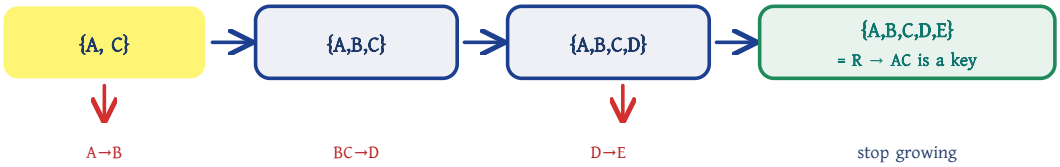
each rung adds one clause to the one below

2 Attribute Closure, Candidate Keys and Superkeys

Attribute closure X^+: The set of every attribute you can derive from X using F repeatedly. **X is a key of R iff $X^+ = R$.** Start with X, then keep adding any Q for which some FD `P → Q` has P already inside the set, until nothing new appears.

```
X⁺ = X
repeat:
  for every FD  P → Q  in F:
      if  P ⊆ X⁺ :  X⁺ ← X⁺ ∪ Q
until  X⁺ stops growing
```

Worked example 1 ✍ the classic closure: `R(A, B, C, D, E)` with `F = {A→B, BC→D, D→E}`. start **{A, C}** → A→B (A is in) → **{A, B, C}** → BC→D (B, C in) → **{A, B, C, D}** → D→E → **{A, B, C, D, E}** = R, so **{A,C}⁺ = {A,B,C,D,E}** and **AC is a key**. With all five attributes in, every FD fires, so it has stopped growing.



the closure ladder: each FD whose left side is already inside pushes one more attribute in

Superkey · candidate key · prime attribute: A **superkey** is any attribute set whose closure is all of R — AB is one above, since {A,B}⁺ = {A,B,C,D,E}. A **candidate key** is a **minimal** superkey: no proper subset still closes R. An attribute is **prime** if it belongs to *some* candidate key.

Worked example 2 ✍ the superkey trap: `R(A, B, C, D)` with `F = {A→B, B→C, C→D}`. **A⁺:** A → {A,B} → {A,B,C} → {A,B,C,D} = R, so A is a superkey; being a **single attribute** it is minimal, so **A is the candidate key**. A also appears on **no right-hand side**, so every key of R must contain A. Then **AB is a superkey but not minimal** (drop B and A alone still closes R), so AB is **not** a candidate key. And **B⁺ = {B, C, D}** — A can never be reached from B.

Exam trap ✍ minimal is the whole question: "Which is the candidate key?" is **not** "which set determines all attributes?" — that is the superkey, and there is usually more than one. Always test **minimality** second: remove one attribute, recompute the closure, and if it is still all of R the set was too big. R(A,B,C,D) above has superkeys AB, AC, AD, ABC, ABD, ACD, ABCD — and exactly **one** candidate key, A.

Memory trick: "**Super is a superset; candidate is the smallest one inside it.**" And "**prime = in some key**", not "in *every* key" — in R(A,B,C) with key A, only A is prime even though A is the sole key. Prime/non-prime is the exact test that separates 3NF from BCNF.

3 First and Second Normal Form

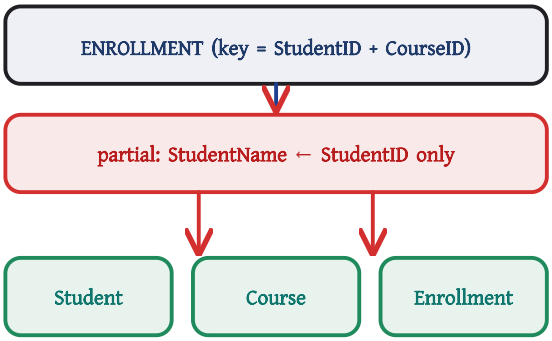
1NF — First Normal Form: Every attribute holds a **single atomic value**. No repeating groups, no lists or arrays inside a cell, no "one row, many facts" columns. A cell is atomic if it cannot usefully be split into several independent values.

Before 1NF	StudentID	Courses
	101	CSE101, MAT201

After 1NF	StudentID	Course
	101	CSE101
	101	MAT201

Exam trap: The comma-separated list breaks **three** rules at once: it is not atomic, it is a repeating group, and it hides a second fact per row. You cannot index `CSE101` without string surgery, and "how many courses does 101 take?" needs a `LIKE` guess. The fix costs **one extra row** and nothing else — always take it.

2NF — Second Normal Form: R is in 2NF iff it is in **1NF** and **no non-prime attribute depends on only part of a composite candidate key**. Partial dependency is impossible when the key is a single attribute, so 2NF only ever bites on **composite-key tables**.



a partial dependency is what forces the split

Worked example ✍️ before 2NF: Key is the composite **(StudentID, CourseID)**.
`(StudentID, CourseID) → StudentName, Instructor, Grade`
Problems: StudentName depends on StudentID **alone** (partial); Instructor depends on CourseID **alone** (partial). Only `→ Grade` needs the whole key, so only it stays behind.

Student	Course	Enrollment
StudentID PK	CourseID PK	StudentID PK, FK
StudentName	Instructor	CourseID PK, FK
		Grade

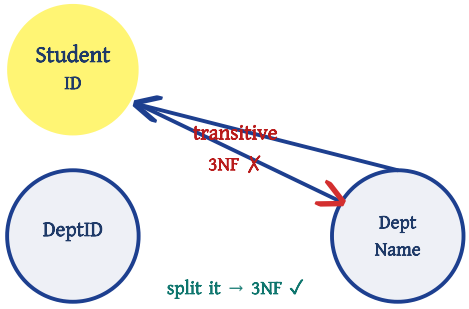
Exam trap ✍️ two: **1.** A partial dependency is defined against a **composite key**; if you are asked about a relation whose key is a single attribute, 2NF is **automatically satisfied** and the real question is 3NF. **2.** The split must be **lossless**: here R1 ∩ R2 = {StudentID}, and StudentID → Student (the whole of R1), so joining the three tables reproduces the original rows exactly — nothing is invented, nothing is lost.

Memory trick: **"Split by the arrow, not by the column."** Draw the FDs; every arrow whose tail is only *half* the key is a partial dependency, and the head of that arrow moves into a table of its own keyed by that tail. 1NF splits by **value** (one fact per cell); 2NF splits by **dependency** (one fact per table).

4 Third and Boyce–Codd Normal Form

3NF — Third Normal Form: R is in 3NF iff it is in **2NF** and no non-prime attribute depends **transitively** on the key: for every FD `X → A` with A non-prime, X must be a superkey. In words: **no attribute may depend on another attribute that is not itself part of the key**.

Worked example ✍️ the genuine 3NF violation: **Before** — `Student(StudentID, Name, DeptID, DeptName)`, key **StudentID**: `StudentID → Name, DeptID` and then `DeptID → DeptName` so `StudentID → DeptID → DeptName`. DeptID is **not** part of the key, yet DeptName hangs off it: a transitive dependency on a non-key attribute. **After** — `Student(StudentID, Name, DeptID·FK)` plus `Department(DeptID, DeptName)`. Now DeptName is stored once per department, no matter how many students sit in it.



the arrow that 3NF is about

BCNF — Boyce–Codd Normal Form: R is in BCNF iff for **every** non-trivial FD `X → Y`, X is a **superkey**. In the shortest exam phrasing: **every determinant must be a candidate key**. It is strictly stronger than 3NF — 3NF only demands this when the dependent is *non-prime*.

Worked example ✍️ a BCNF violation that 3NF cannot remove: Consider `R(CourseID, Semester, Instructor)` with `(CourseID, Semester) → Instructor` and `Instructor → CourseID`. Instructor is a prime attribute — it sits in no candidate key here, but the relation is **still only 2NF**, because the determinant `Instructor` is not a superkey. Check each side: neither **CourseID** alone nor **Semester** alone closes the relation, so the determinant (CourseID, Semester) is not a superkey either. **BCNF solution:** split — `Instructor(CourseID, Instructor)` keyed by Instructor, and `Offering(CourseID, Semester)`.

Exam trap ✍️ three: **1.** 3NF says "every *non-prime* attribute depends on the key"; BCNF says "every *determinant* is a key". A relation can satisfy 3NF and still fail BCNF exactly when a transitive chain runs through a **prime** attribute. **2.** The single-attribute key case has no partial dependency at all, so it can be in 2NF while still failing 3NF — the two rungs test different things. **3. BCNF is not the top of the ladder**: a 4NF exists for multi-valued dependencies. "The highest normal form" in an MCQ still means BCNF.

Memory trick: "**1NF one value, 2NF whole key, 3NF no detour, BCNF key on every arrow.**" Read BCNF as a rule about the *left-hand sides*: walk through every FD in F and ask "is this determinant a key?" If any answer is no, split that relation.

5 The Ladder in One Table — and How Far Up It Goes

Normal form	Requirement	In plain words
1NF	atomic values, no repeating groups	one fact per cell
2NF	1NF + **no partial dependency**	nothing hangs off half a composite key
3NF	2NF + **no transitive dependency**	nothing hangs off a non-key attribute
BCNF	3NF + **every determinant is a candidate key**	every arrow starts at a key

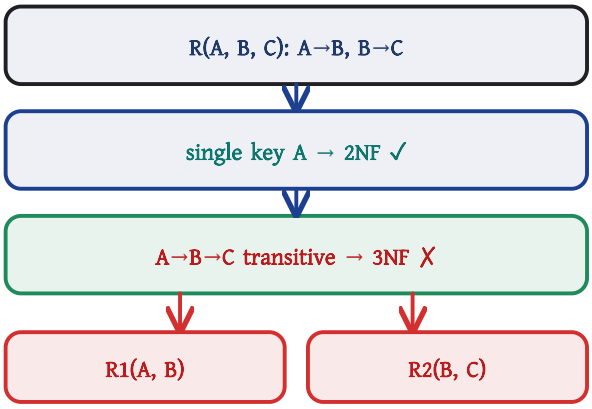
Worked example ✍️ "the highest normal form": `R(A, B, C)` with `F = {A→B, B→C}`, and A its only candidate key.

Keys. A appears on no right-hand side, so every key contains A. A^+ = {A, B, C} = R, and A is one attribute, so A is the **sole candidate key** and the only **prime** attribute.

2NF? The key is a **single** attribute, so no partial dependency can exist. **R is in 2NF.**

3NF? A→B→C is transitive: in B→C, B is not a superkey and C is not prime. So R violates 3NF and BCNF too.

Answer: 2NF. Decompose: `R1(A, B)`, `R2(B, C)`.



why 2NF and not 3NF

How to answer any "highest normal form" question: Four moves, in order: **1.** find the candidate keys (closures + minimality); **2.** list the prime attributes; **3.** if the key is composite, look for **partial** dependencies — if any, stop at 1NF; **4.** otherwise look for a **transitive** dependency on a non-key attribute — if any, stop at 2NF; if none but some determinant is not a key, stop at 3NF; otherwise **BCNF**.

Memory trick for the whole ladder: "**Atomic, All-key, Away-from-key, BCNF.**" 1NF is about *values*, 2NF about *the key's parts*, 3NF about *non-key detours*, BCNF about *the determinants themselves*. Each rung is "the one below, plus one more clause" — so a relation in BCNF is automatically in all three below it, never the reverse.

Exam trap ✍️: Students often answer "3NF" whenever a transitive dependency exists, forgetting that **3NF only forbids a transitive dependency whose destination is non-prime**. And they often call AB the candidate key when A alone already closes the relation — **minimality**, not size, is the test. Check prime membership before you write "3NF".

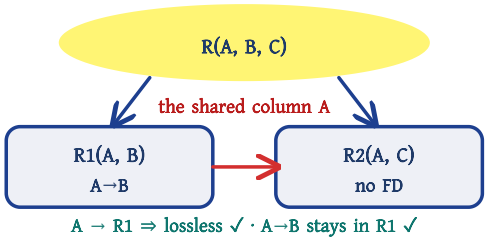
6 Decomposition: Lossless Join and Dependency Preservation

Splitting a relation is a legitimate normalisation only if it is **lossless** (no information invented or destroyed) and **dependency-preserving** (the FDs can still be checked without joining).

The lossless-join rule: For R → R1, R2 the join is **lossless** iff

`(R1 ∩ R2) → R1` or `(R1 ∩ R2) → R2`

Read it as: the attributes the pieces **share** must be a key of at least one of them. If the shared attribute decides one whole side, every original tuple has a definite partner and nothing is fabricated or lost.



the shared attribute decides one whole side

Dependency preservation: The decomposition is **dependency-preserving** iff the union of the projections of F onto R1 and onto R2 is exactly F — in practice, iff **every FD in F lives entirely inside one piece** and can be enforced there alone, with no join.

Worked example ✍️ the standard two-part answer: `R(A, B, C)` with `F = {A→B}`, decomposed into `R1(A, B)` and `R2(A, C)`.
Lossless? `R1 ∩ R2 = {A}`, and `A→B` means `A→R1`, so the common attribute A is a key of R1. The first alternative holds → **lossless**: every original tuple has one A-value, hence exactly one R1 row and one R2 row to pair.
Preserved? The only FD in F is `A→B` and it lies entirely inside R1, so it can be enforced there with no join → **dependency-preserving**.

Exam trap ✍️ the lossy counter-example: Take the same R(A, B, C) with F = {A→B} but decompose it wrongly, into `R1(A, C)` and `R2(B, C)`. Now R1 ∩ R2 = {C}, and C determines neither R1 nor R2, so the rule fails: the join is **lossy** — pairing C = 3 from R1 with C = 3 from R2 produces a row (3, 3) that was never in R. The pieces must be split so the shared attribute is a **key** on one side; sharing a mere foreign key is not enough.

Exam trap ✍️ lossless ≠ preserved: The two tests answer different questions and a decomposition can pass one and fail the other. A split can be lossless yet force you to **join** to check `A→B`, leaving the database unable to police the constraint. A split can preserve every dependency yet be lossy — which silently corrupts data. Give both verdicts separately.

Memory trick: "**Lossless asks about the overlap; preservation asks about the arrows.**" Overlap: the shared columns must decide one whole side. Arrows: every FD must sit inside one side. Neither test needs the data — **the FDs alone decide it**, which is why both can be examined in a written paper.

7 The Whole Normalization Example, Step by Step

StudentID	Name	DeptID	DeptName	CourseID	CourseName	Grade
101	Alice	10	CSE	C101	DBMS	A
101	Alice	10	CSE	C102	Algorithms	B

What is wrong with this one table: It is already in 1NF — every cell holds a single value. But the key is the composite (StudentID, CourseID), and the table repeats three facts: **Alice** is written twice, **10 / CSE** is written twice, and a course name would be written once per student. A student's name and a department's name each live in several rows, and any one of those rows can disagree with the others.

The three anomalies it causes: **Update** — rename Alice and you must edit every row she appears in; miss one and the database holds two names for student 101. **Insert** — a new department with no student cannot be recorded, because there is no StudentID to put beside it. **Delete** — delete Alice's only enrolment row and you lose her name *and* the fact that department 10 exists.

Step 1 — 1NF ✍️: Already satisfied: each column holds one atomic value and the grade belongs to the pair (student, course), not to the student. Nothing to do.
Step 2 — 2NF ✍️ The key (StudentID, CourseID) is composite, and three non-key attributes depend on only **half** of it: `StudentID → Name`, `CourseID → CourseName` and `StudentID → DeptID` are all partial, while `(StudentID, CourseID) → Grade` is full. Split each partial dependency into its own table.
Step 3 — 3NF ✍️ The genuine remaining violation is the transitive chain `StudentID → DeptID → DeptName` inside `Student(StudentID, Name, DeptID, DeptName)`: DeptID is **not** part of the key, yet DeptName hangs off it. Moving DeptName out into its own Department table is exactly what removes it — that is the whole content of the 3NF step, and it is why Department must appear in the final design.

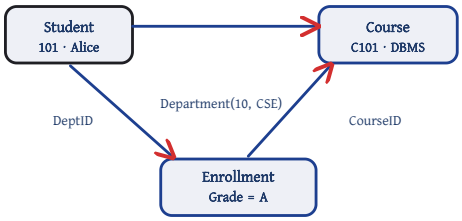
Student	Name	DeptID
101	Alice	10

Department	DeptName
10	CSE

Course	CourseName
C101	DBMS

Enrollment	Grade
101	A

Keys underlined; DeptID and both IDs of Enrollment are foreign keys into the tables above.



one fact, one place

The final design, and what it buys: Four relations: **Student(StudentID, Name, DeptID)** · **Department(DeptID, DeptName)** · **Course(CourseID, CourseName)** · **Enrollment(StudentID, CourseID, Grade)**. Grade stays with the pair it describes, a department's name is stored **once**, a course name is stored **once**. Renaming a student is now a **single** row update, a new empty department can be inserted without inventing a student, and deleting an enrolment cannot take a department with it.

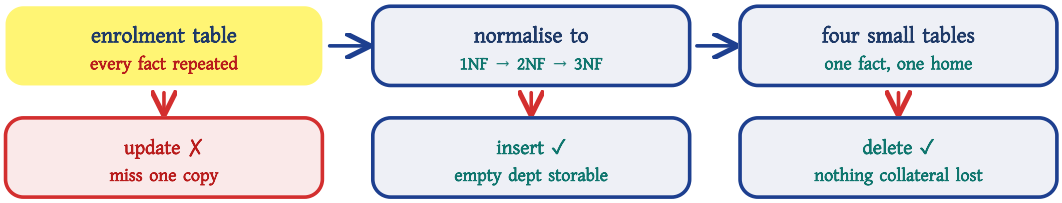
8 Anomalies, Denormalization and the Trade-off

The three anomalies, named: An **anomaly** is a wrong update that becomes possible only because the same fact is stored in several places. There are exactly three kinds, and a written question usually wants all three by name.

Update anomaly ✍️: A fact must be changed in **every** copy or the relation becomes self-contradictory. Changing Alice's name means editing each row where 101 appears; one forgotten row leaves two names for the same student, and nothing in the schema stops it.

Insert anomaly ✍️: A perfectly valid fact **cannot be stored** because the key demands a companion fact that does not exist yet. A brand-new department with no students has no StudentID to accompany it.

Delete anomaly ✍️: Deleting one row silently removes facts that had nothing to do with it. Removing Alice's only enrolment row deletes her name and the existence of department 10 as well.



why the split is worth making

Benefits of normalisation: **1.** Less redundancy, so less disk and fewer copies to keep in step. **2.** Better integrity — each fact has one home, so a single update is the whole update. **3.** Fewer update, insert and delete anomalies. **4. Smaller, more meaningful tables**, so indexes and joins are cheaper to plan.

Denormalization — the deliberate undo: **Denormalization** is **intentionally** re-introducing redundancy after normalising so that **reads** get fast. A normalised schema needs a three-table join to print one report row; copying a department name into a fact table turns that join into a single scan. It is a deliberate trade — **write once, read a million times** — and it re-opens all three anomalies.

Exam trap ✍ the direction question: "Which is better?" has no answer — it depends on the workload. Normalise for **OLTP** (many small inserts, updates and deletes, integrity enforced by the DBMS); denormalise for **reporting / data warehousing** (huge read volumes, occasional batch reloads). "Denormalization always improves performance" is **false**: it speeds reads and slows every write.

Where the rest of this chapter lives: **The algebra and SQL that read these tables** were covered in **db2**; **relational and ER models, constraints** in **db1**. **File organisation, indexes, B+ trees, cost-based optimisation** → **db4**. **Transactions, ACID, isolation, concurrency, recovery** → **db5**.

Night-Before-the-Exam Sheet

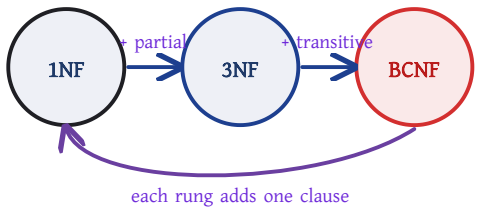
Fact sheet:

$X \to Y$: same $X \Rightarrow$ same Y	X = determinant, Y = dependent
Trivial: $Y \subseteq X$ · Non-trivial: $Y \not\subset X$	Partial: on part of X · Transitive: $A \to B$, $B \to C$
X^+ = everything derivable from X	X is a key iff $X^+ = R$
Superkey = any key	Candidate key = minimal superkey
Prime = in some candidate key	1NF: atomic values
2NF: + no partial dependency	3NF: + no transitive dependency
BCNF: + every determinant is a key	Anomalies: update, insert, delete
Lossless $\Leftrightarrow$ $(R1 \cap R2) \to R1$ or $\to R2$	Preserved $\Leftrightarrow$ every FD inside one piece
$\{A,C\}^+$ in $R(A,B,C,D,E)$ = all five	Key of $R(A,B,C,D)$ with $A \to B \to C \to D$ is A alone
$R(A,B,C)$, $A \to B \to C$, key $A \Rightarrow$ 2NF only	Normalise for writes, denormalise for reads

Top 12 one-liners for MCQs

1. $A \to B$: two rows agreeing on A agree on B
2. Trivial FD: $Y \subseteq X$ — always true, never interesting
3. $X^+ = R$ means X is a key
4. Superkey = any key; candidate = the minimal one
5. Prime attribute = belongs to some candidate key
6. 1NF needs one atomic value per cell
7. 2NF matters only for composite keys
8. 3NF forbids transitives through non-key attributes
9. BCNF: every determinant must be a candidate key
10. Lossless $\Leftrightarrow$ $(R1 \cap R2) \to R1$ or $(R1 \cap R2) \to R2$
11. Lossless is about the overlap, preservation about the arrows
12. The anomalies are update, insert and delete

Top traps: calling a **superkey** the candidate key · forgetting to test **minimality** · answering "3NF" whenever a transitive dependency exists, without checking **primeness** · applying 2NF to a single-attribute key · reading the arrow backwards · omitting the **composite key** in the 2NF "before" table · claiming a decomposition is safe without testing **lossless-join** · confusing lossless with dependency-preserving · treating denormalization as free.



one clause per rung

All tricks together: An arrow is a promise · one value per cell, then one arrow per table · split by the arrow, not by the column · super is a superset, candidate the smallest one inside it · atomic, whole key, no detour, key on every arrow · lossless asks about the overlap, preservation about the arrows · normalise for writes, denormalise for reads.

✓ Practice MCQs (exam pattern, 10 marks)

1. A relation is in First Normal Form (1NF) if:
(a) Every attribute holds only atomic values (b) It has no partial dependencies
(c) It has no transitive dependencies (d) Every determinant is a candidate key

2. `R(A, B, C, D)` has `A→B`, `B→C`, `C→D`. The candidate key of R is:
(a) B (b) AB
(c) A (d) CD

3. `R(A, B, C, D, E)` has `A→B`, `BC→D`, `D→E`. The closure $\{A, C\}^+$ is:
(a) {A, B, C} (b) {A, B, C, D}
(c) {A, C, E} (d) {A, B, C, D, E}

4. `R(A, B, C)` has `A→B`, `B→C`, and A is its only candidate key. The highest normal form is:
(a) 1NF only (b) 2NF
(c) 3NF (d) BCNF

5. `R(A, B, C)` with `F = {A→B}` is split into `R1(A, B)` and `R2(A, C)`. Which is correct?
(a) Lossy but dependency-preserving (b) Neither lossless nor preserving
(c) Lossless but not dependency-preserving (d) Lossless and dependency-preserving

6. In `Enrollment(StudentID, CourseID, Grade, StudentName, Instructor)` keyed (StudentID, CourseID), which FD is partial?
(a) StudentID→StudentName (b) (StudentID, CourseID)→Grade
(c) CourseID→Instructor (d) All three

7. A relation is in BCNF only if:
(a) It has no transitive dependencies (b) Every determinant is a candidate key
(c) It has no partial dependencies (d) Every attribute holds an atomic value

8. For R split into R1 and R2, the join is lossless iff:
(a) R1 ∩ R2 is empty (b) R1 and R2 are both in 3NF
(c) (R1 ∩ R2)→R1 or (R1 ∩ R2)→R2 (d) Every dependency is preserved

9. Denormalization is deliberately used to:
(a) Remove redundancy and eliminate anomalies (b) Speed up read-heavy queries by accepting redundancy
(c) Enforce foreign-key constraints automatically (d) Reduce update anomalies

10. In `R(A, B, C)` with `A→B`, `B→C` and A the only candidate key, how many prime attributes are there?
(a) 1 (b) 2
(c) 3 (d) 0

Answer key → 1-a · 2-c · 3-d · 4-b · 5-d · 6-a · 7-b · 8-c · 9-b · 10-a

Written question: Define a functional dependency and name the five types, distinguishing trivial from non-trivial.

For `R(A, B, C, D, E)` with `F = {A→B, BC→D, D→E}`, compute $\{A, C\}^+$ and say whether AC is a candidate key.

For `R(A, B, C)` with `A→B`, `B→C` and A the only candidate key, state the highest normal form and decompose it.

Then test the split of that R into `R1(A, B)` and `R2(A, C)` for lossless-join and dependency preservation.

10 marks

1. **Definitions (2)** — `X → Y` holds when every two tuples agreeing on X also agree on Y; X is the determinant. **Trivial**: Y ⊆ X, e.g. (A, B)→A. **Non-trivial**: Y ⊄ X, e.g. StudentID→Name. **Full**: Y depends on the whole of X. **Partial**: on part of X. **Transitive**: A→B and B→C give A→C. 2
2. **Closure (2)** — {A, C}; A→B adds B → {A, B, C}; BC→D adds D → {A, B, C, D}; D→E adds E → **{A, B, C, D, E} = R**, so **AC is a candidate key**. 2
3. **Highest normal form (2)** — A is on no right-hand side and is one attribute, so it is the sole candidate key and the only prime attribute. A single key means no partial dependency, so R is in **2NF**. But A→B→C is transitive, and in B→C the determinant is not a superkey and C is not prime, so 3NF fails: the answer is **2NF**, decomposed to **R1(A, B)** and **R2(B, C)**. 2
4. **Lossless (2)** — R1 ∩ R2 = {A}, and A→B gives A→R1, so the shared attribute is a key of R1: the join is **lossless**. 2
5. **Preservation + rules (2)** — the only FD, A→B, lies entirely inside R1, so it holds with no join: **dependency-preserving**. In general lossless ⟺ (R1∩R2)→R1 or (R1∩R2)→R2; preserving ⟺ the projections of F onto R1 and R2 union to F. 2

Before the exam ✓

- ✓ Can write a definition and the five FD types
- ✓ Can compute X^+ and find candidate keys
- ✓ Can state the four normal-form rules
- ✓ Can test a decomposition for lossless-join

✓ Can separate a superkey from a candidate key

✓ Can name the anomalies and denormalization